

Лабораторная 4. Слияние, подсчёт, разряды и пирамида

Почему нужны разные сортировки

Один алгоритм может экономить память, другой — сохранять порядок равных записей, третий — использовать небольшой диапазон целых ключей. Эта работа сравнивает MergeSort, CountingSort, RadixSort, HeapSort, QuickSort из ЛР 3 и встроенный sorted.[1] Собственные сортировки изменяют переданный список; sorted возвращает новый.

MergeSort: сначала половины, затем слияние

Массив делится пополам до участков длиной 0 или 1. Такие участки уже отсортированы. Затем две отсортированные половины объединяются: сравниваются первые ещё не взятые элементы, меньший помещается в выход. Когда одна половина закончилась, остаток другой можно перенести без сравнений ключей.

Слияние [1,4] и [2,3] последовательно выбирает 1,2,3,4. Инвариант: уже записанная часть выхода содержит наименьшие из объединяемых элементов в правильном порядке; i и j указывают на первые ещё не использованные значения. Если ключи равны, берём левый элемент: именно условие $\leq$ сохраняет исходный порядок равных записей.

Рекуррент $T(n)=2T(n/2)+\Theta(n)$ даёт $\Theta(n \log n)$. Дополнительный буфер длины n используется повторно во всех слияниях, стек имеет $O(\log n)$ кадров, общая дополнительная память $O(n)$. Эта реализация не пропускает слияние уже упорядоченных половин, поэтому её лучший случай тоже $\Theta(n \log n)$.

CountingSort: распределение по целым ключам

Если ключи — целые числа из небольшого диапазона, сравнивать каждую пару не обязательно. Пусть $\text{minimum}=-2$, $\text{maximum}=2$. Для пяти возможных ключей создаются пять счётчиков. Ключ x относится к ячейке $x-\text{minimum}$; так поддерживаются отрицательные значения.

Для $[-2,1,-2,0]$ частоты равны $[2,0,1,1,0]$, а префиксные суммы — $[2,2,3,4,4]$. Префиксная сумма показывает, сколько элементов не превосходит данный ключ. Она задаёт границу его блока в отсортированном результате.

Вход просматривается справа налево. Перед размещением элемента соответствующая сумма уменьшается, и элемент занимает найденную позицию. Более поздняя равная запись попадает правее, поэтому порядок одинаковых ключей сохраняется. Простой вывод ключа `count` раз отсортировал бы числа, но потерял бы связанные метки записей.

Время $O(n+K)$, дополнительная память $O(n+K)$, где $K=\text{max}-\text{min}+1$ — количество возможных целых значений. В формулировке КИМ $k=\text{max}-\text{min}$ отличается на единицу и даёт ту же асимптотику. При $n=1000$ и диапазоне до 10^{12} выделение массива счётчиков бессмысленно. Реализация явно отклоняет диапазоны шире 2 000 001, а не пытается занять огромную память.

RadixSort LSD: сортируем цифру за цифрой

LSD начинает с младшего разряда. Например, [21,13,12] после устойчивого прохода по единицам превращается в [21,12,13], а после десятков — в [12,13,21]. Порядок внутри равной десятки должен сохранить уже установленный порядок единиц. Поэтому каждый внутренний проход обязан быть устойчивым.[2]

В коде используется десятичная counting-сортировка с десятью корзинами. К каждому ключу применяем одинаковое смещение: $\text{key} - \text{minimum}$. Это превращает отрицательные ключи в неотрицательные и сохраняет порядок, поскольку $x < y$ тогда и только тогда, когда $x - \text{minimum} < y - \text{minimum}$. Сами записи не изменяются, меняются только вычисленные ключи.

Если d — число десятичных разрядов наибольшего смещённого ключа, время $O(d(n+10))$, память $O(n+10)$. При одинаковых ключах проходы не нужны. Для произвольно длинных Python int операции получения цифр тоже имеют стоимость: объявлять такую сортировку безусловно $O(n)$ для любых целых нельзя.

HeapSort: максимум всегда наверху

Куча хранится прямо в массиве. У узла i дети имеют индексы $2i+1$ и $2i+2$. В максимальной куче ключ родителя не меньше ключей детей. Это локальное свойство не означает, что весь массив отсортирован.

Просейка вниз сравнивает узел с большим ребёнком. Если ребёнок больше, они меняются, и проверка продолжается ниже. Куча строится от последнего внутреннего узла к корню: каждый ребёнок уже корень правильной подкучи.

Почему построение $O(n)$, хотя отдельная просейка $O(\log n)$? Половина узлов — листья и не двигается, четверть может опуститься на один уровень, восьмая часть — на два. Сумма ограничена $n \cdot \sum (h/2^{(h+1)}) = O(n)$. Умножать n на максимальную высоту — допустимая, но слишком грубая верхняя оценка.

Далее максимум меняется с последним элементом активной кучи. Размер кучи уменьшается, новый корень просеивается вниз. Суффикс после активной области уже отсортирован. Выполняется $O(n \log n)$ работы и используется $O(1)$ дополнительной памяти. Нестабильность связана с дальними обменами, способными переставить одинаковые ключи.

Сводная таблица для реализованного кода

Здесь время оценивается в модели операций с ключами ограниченной длины; K — ширина диапазона, d — число разрядов. Для среднего случая QuickSort предполагается случайный выбор опорного элемента.

Алгоритм	Лучший	Средний	Худший	Доп. память	Стабилен	Ограничения
MergeSort	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n \log n)$	$O(n)$	Да	Сравнимые ключи
CountingSort	$\Theta(n+K)$	$\Theta(n+K)$	$\Theta(n+K)$	$O(n+K)$	Да	Целые, ограниченный K
Radix LSD	$\Theta(n+d(n+10))$	$\Theta(n+d(n+10))$	$\Theta(n+d(n+10))$	$O(n+10)$	Да	Целые; d зависит от ключей

Алгоритм	Лучший	Средний	Худший	Доп. память	Стабилен	Ограничения
HeapSort	$\Theta(n)$ на равных	$\Theta(n \log n)$	$\Theta(n \log n)$	$O(1)$	Нет	Сравнимые ключи
QuickSort, 3 части	$\Theta(n)$ на равных	$O(n \log n)$, ожидаемо	$O(n^2)$	$O(\log n)$	Нет	Сравнимые ключи
sorted, Timsort	$\Theta(n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Да	Сравнимые ключи

Для HeapSort лучший линейный случай указан именно для равных ключей в данной просейке с ранней остановкой. На различных ключах типичный анализ сортировочной фазы даёт $\Theta(n \log n)$. Python гарантирует стабильность sorted и позволяет задавать отдельную функцию key.[3]

Почему нижняя граница не запрещает CountingSort

Сортировку только сравнениями различных ключей можно представить деревом решений. Чтобы различить $n!$ возможных порядков, двоичное дерево должно иметь хотя бы $n!$ листьев; его высота не меньше $\log_2(n!) = \Omega(n \log n)$. Это нижняя граница для модели сравнений.

CountingSort обращается в ячейку по численному значению ключа; RadixSort извлекает цифры. Они используют дополнительную информацию о представлении данных и не ограничены той же моделью. Платой становятся требования к диапазону, разрядам и памяти.

Запуск и проверка

```
python -m pytest labs/lab04 -q
python -m labs.lab04.experiment --variant 15
```

Эксперимент включает пять размеров 1000...100000 и пять классов: широкий случайный диапазон, узкий диапазон с повторами, прямой порядок, обратный и почти отсортированные данные с 1% случайных обменов. Все алгоритмы получают одинаковый вход. Копирование исходного массива выполняется до таймера; собственные выделения внутри алгоритма учитываются.

У QuickSort сохранены счётчики из ЛР 3, а у новых сортировок их нет. Поэтому сравнение времени относится к данным конкретным реализациям, и постоянные затраты QuickSort выше. Встроенный sorted выполняется в скомпилированном коде; его выигрыш нельзя объяснять одной только асимптотикой.

Вопросы на защите и ответы

1. Что обеспечивает стабильность MergeSort? При равенстве ключей выбирается элемент левой половины. Стабильность обеих половин сохраняется индуктивно при их слиянии.

2. Когда CountingSort проигрывает? Когда K велик относительно n . Нужно инициализировать и обойти K счётчиков, даже если большинство значений отсутствует.

3. Чем Radix отличается от Counting? Counting выделяет корзины на весь диапазон; Radix многократно использует фиксированное количество корзин для одной цифры. Число проходов зависит от d .

4. Почему внутренняя сортировка LSD должна быть устойчивой? Следующий разряд уточняет порядок, сохраняя порядок по уже обработанным младшим цифрам внутри равных старших. Неустойчивый проход эту информацию уничтожит.

5. Почему HeapSort нестабилен? Обмен корня с последним элементом может переместить равный ключ через другой равный ключ. Свойство кучи контролирует величины, но не исходные метки.

6. Как доказать $\Omega(n \log n)$? Посчитать $n!$ возможных перестановок и минимальную высоту дерева двоичных сравнений. Граница относится к худшему случаю для произвольных различных сравнимых ключей.

7. Как сортируются отрицательные числа по разрядам? Вычитается общий минимум, а устойчивые проходы работают со смещёнными неотрицательными ключами. Общий сдвиг не меняет отношение порядка.

8. Что означает in-place в этой работе? HeapSort меняет исходный массив без вспомогательного массива длины n . Локальные индексы и временная ссылка обмена занимают $O(1)$.

Источники

[1] МТУСИ. КИМ-04, 2026.

[2] Robert Sedgewick, Kevin Wayne. String Sorts: LSD, материалы авторов Algorithms, 4th edition.

[3] Andrew Dalke, Raymond Hettinger. Sorting Techniques, Python Software Foundation.

Отчёт по ЛР 4. Эффективные сортировки

Цель и выполненная работа

Реализованы алгоритмы и проверки, описанные в конспекте. Эксперимент выполнен на данных варианта 15, seed 45. В таблицах приведены фактические результаты запуска; время указано в секундах, если не оговорено иное.

Условия и методика

CPU: 12th Gen Intel(R) Core(TM) i7-12650H. ОС: Windows-11-10.0.26200-SP0. Python 3.12.12, NumPy 2.5.3, Matplotlib 3.11.1. Дата и время UTC: 2026-09-10T18:29:14.393722+00:00.

Один прогрев и пять повторов на точку, итог — медиана `time.perf_counter`. Ввод, генерация и подготовка свежего состояния исключены из таймера. Фоновые процессы и питание не контролировались; задержки рабочего компьютера могут влиять на отдельные точки. Контрольные суммы данных находятся в `environment.json`.

Результаты и их объяснение

Вход	Алгоритм	K	Медиана, с
wide	merge_sort	1999940	0.459073
wide	counting_sort	1999940	0.315174
wide	radix_sort	1999940	0.329488
wide	heap_sort	1999940	0.671727
wide	quick_sort	1999940	1.10013
wide	sorted	1999940	0.0180467
narrow	merge_sort	1000	0.457632
narrow	counting_sort	1000	0.0364277
narrow	radix_sort	1000	0.114914
narrow	heap_sort	1000	0.621409
narrow	quick_sort	1000	0.603698
narrow	sorted	1000	0.0150966
sorted	merge_sort	1999940	0.387641
sorted	counting_sort	1999940	0.274492
sorted	radix_sort	1999940	0.291561
sorted	heap_sort	1999940	0.621497
sorted	quick_sort	1999940	1.00408
sorted	sorted	1999940	0.001495
reversed	merge_sort	1999940	0.394731
reversed	counting_sort	1999940	0.26325
reversed	radix_sort	1999940	0.278887
reversed	heap_sort	1999940	0.580826
reversed	quick_sort	1999940	1.0085
reversed	sorted	1999940	0.0036791
nearly	merge_sort	1999940	0.434107
nearly	counting_sort	1999940	0.258622
nearly	radix_sort	1999940	0.27368
nearly	heap_sort	1999940	0.639326
nearly	quick_sort	1999940	0.993363
nearly	sorted	1999940	0.0035861

Широкий диапазон делает CountingSort дорогим даже при небольшом n: требуется обойти почти два миллиона счётчиков. На узком диапазоне объём этой работы существенно меньше. MergeSort и HeapSort не используют числовой диапазон, RadixSort зависит от количества разрядов.

Встроенный sorted выполняется в С и использует уже имеющуюся упорядоченность. QuickSort перенесён из ЛР 3 со счётчиками, новые сортировки работают без них; сравнение показывает конкретные реализации и не изолирует только асимптотику. Теоретическая таблица времени, памяти и стабильности приведена в конспекте.

На $n=100000$ для wide минимальная медиана у sorted: 0.0180467 с. Это вывод по данной точке, а не универсальный рейтинг.

На $n=100000$ для narrow минимальная медиана у sorted: 0.0150966 с. Это вывод по данной точке, а не универсальный рейтинг.

На $n=100000$ для sorted минимальная медиана у sorted: 0.001495 с. Это вывод по данной точке, а не универсальный рейтинг.

На $n=100000$ для reversed минимальная медиана у sorted: 0.0036791 с. Это вывод по данной точке, а не универсальный рейтинг.

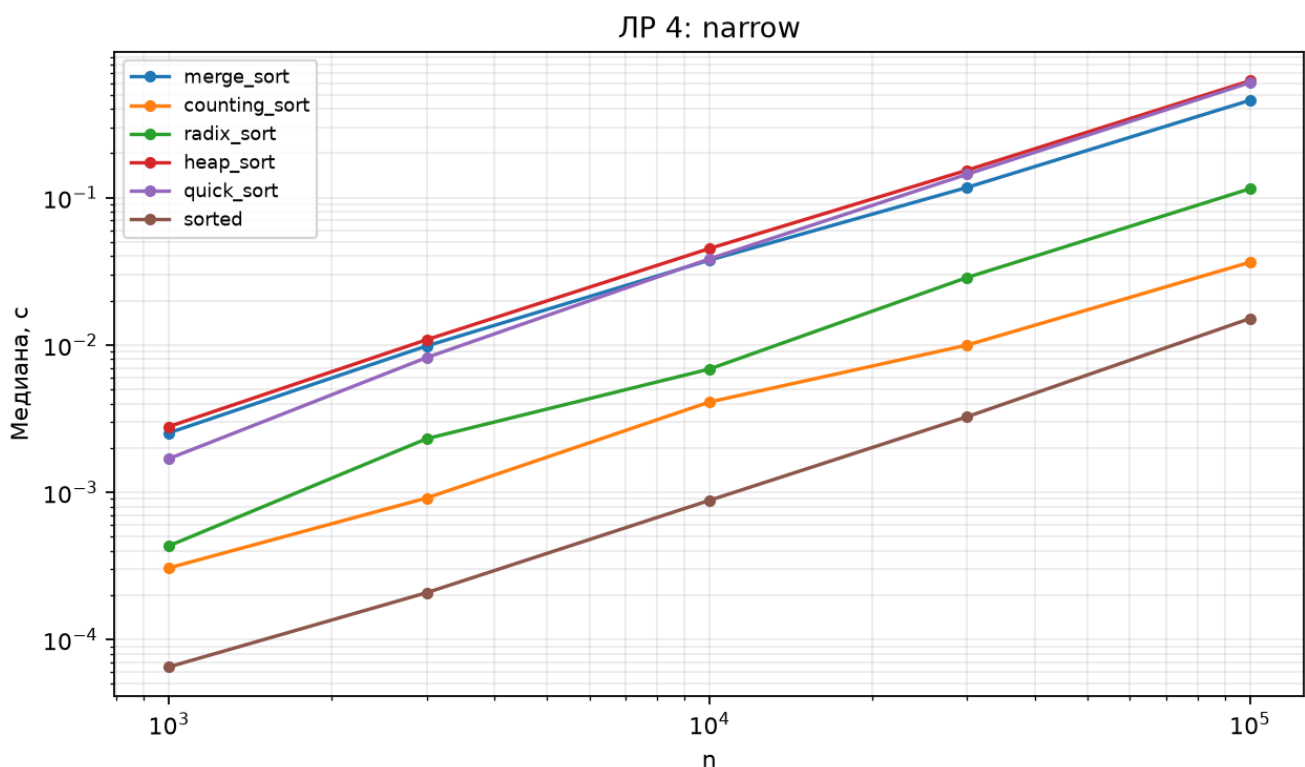
На $n=100000$ для nearly минимальная медиана у sorted: 0.0035861 с. Это вывод по данной точке, а не универсальный рейтинг.

Проверка и вывод о применимости

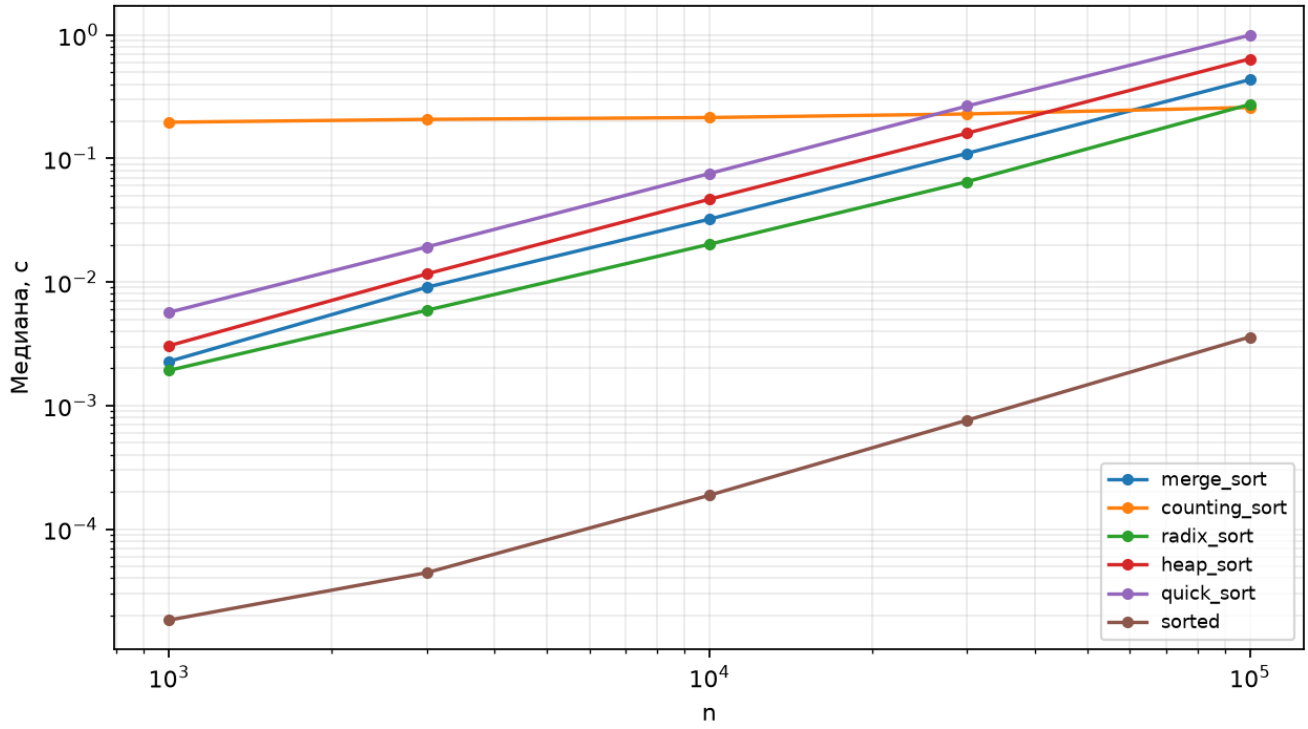
Проверки находятся в test_lab.py. Запуск: `python -m pytest labs/lab04 -q`. Применены граничные случаи, инварианты и независимые эталоны; состав тестов подробно разобран в конспекте. Проверки всего комплекта также сохранены в verification.txt в корне репозитория.

Вывод: принять для учебного использования в рамках явно описанных контрактов. Измерения подтверждают основные тенденции роста; ограничения реализаций и сравнения указаны выше и в конспекте. Автоматическая проверка не оценивает готовность к устной защите.

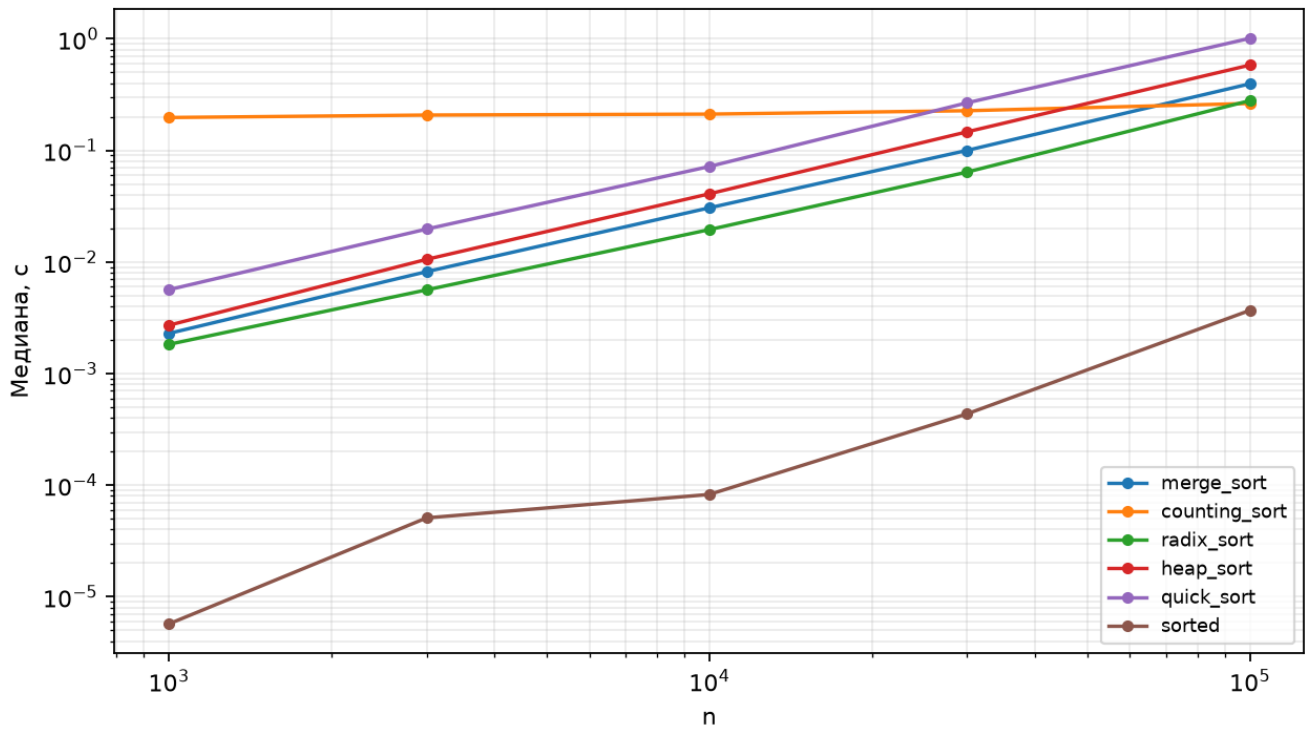
Графики

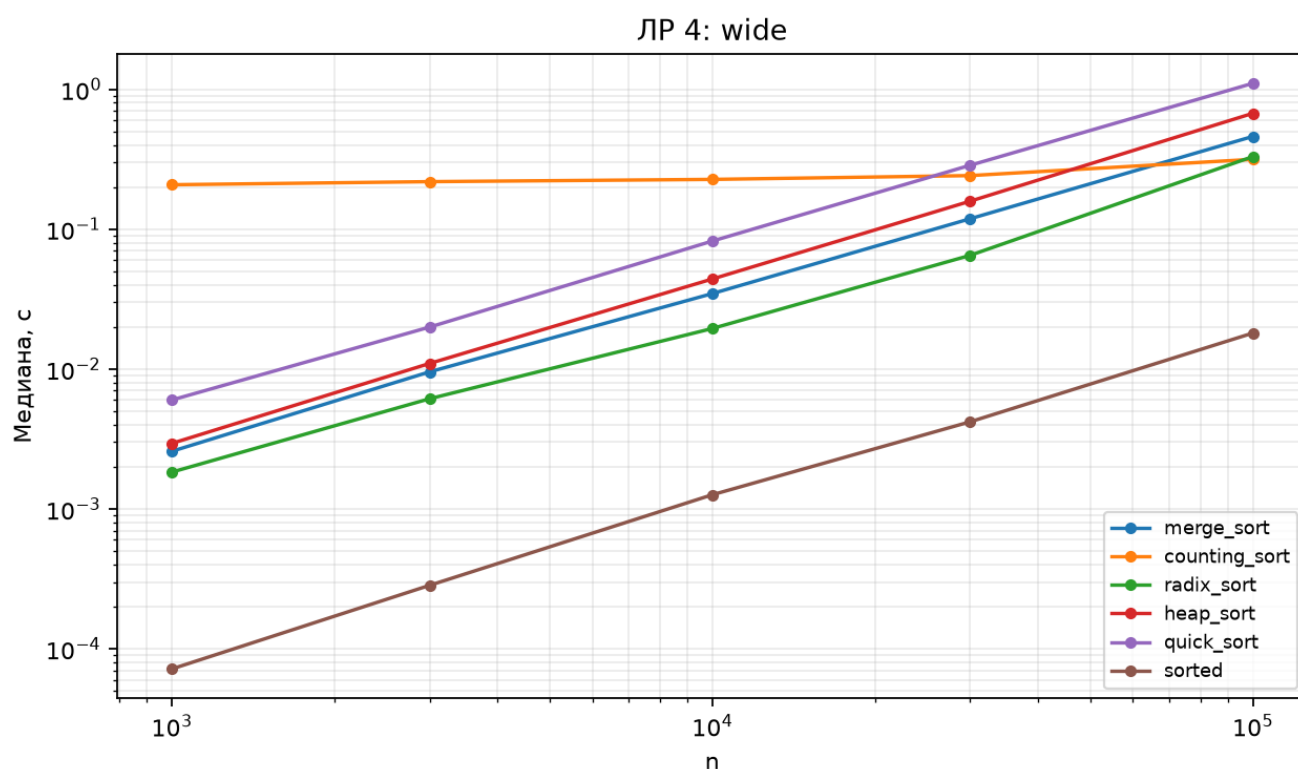
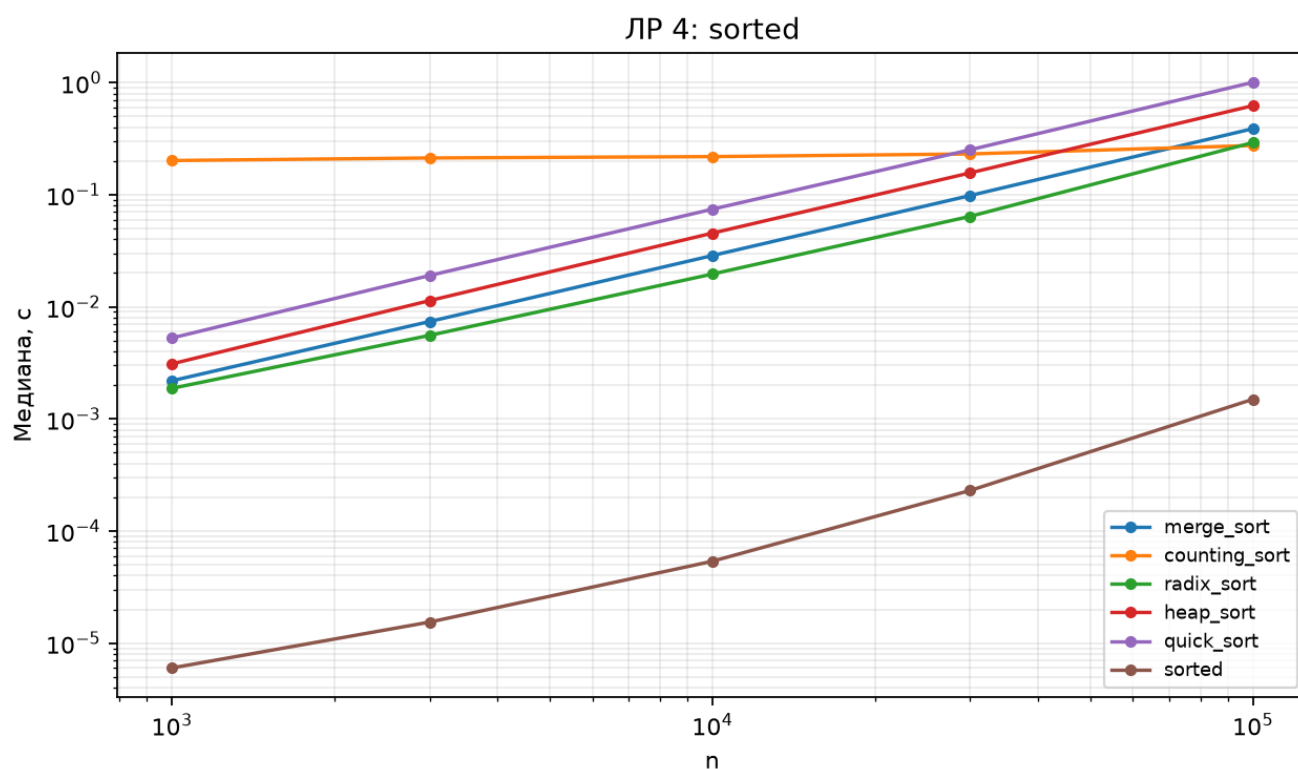


ЛР 4: nearly



ЛР 4: reversed





Исходные данные отчёта

CSV содержит медиану и все пять длительностей trial_1...trial_5. Вспомогательные таблицы без времени содержат точные счётчики или свойства структуры.

- measurements.csv
- Паспорт окружения и контрольные суммы